

Builder Portfolio Management System

Problem Statement:

The Builders' Association aims to transition from manual, spreadsheet-based project record management to a robust, real-time, console-based Java application. The current manual process leads to data inconsistency, loss, errors, and inefficient communication with clients and stakeholders. To address these issues, you are tasked with designing and developing a Builder Portfolio Management System that supports real-time data handling, multiple user roles, and reliable project management functionalities.

Proposed Solution

To solve the issues of manual spreadsheet-based management, we designed and implemented a **console-based Builder Portfolio Management System** using a **layered architecture** (Presentation, Service, Repository, Model).

The system introduces **role-based access control** with four user roles: Admin, Manager, Builder, and Client. Authentication and authorization are handled through a dedicated `AuthenticationService`.

All business logic is isolated within the **Service Layer**, ensuring proper validation, role checks, and controlled operations. Data is stored in-memory using **ConcurrentHashMap** to ensure thread safety and real-time data handling.

Admins manage projects and assign managers. Managers create and assign tasks to builders while maintaining project progress. Builders update task statuses, and clients can view and purchase available projects.

The design follows **Separation of Concerns, SRP, and modular principles**, ensuring scalability, maintainability, and reduced data inconsistency.

This structured approach eliminates manual errors, improves reliability, and enables efficient project lifecycle management.

1.High-Level Design (HLD)

1.1 System Overview

The **Builder Portfolio Management System** is a console-based Java application designed to manage construction projects, builders, managers, and clients in a structured and role-based manner.

The system replaces manual spreadsheet-based tracking with a layered architecture system that ensures:

- Role-based authentication
- Project lifecycle management
- Task assignment and tracking
- Builder management
- Client investment tracking

1.2 Actors & Roles

Role	Description
Admin	Manages projects and assigns managers
Manager	Manages builders and tasks
Builder	Works on assigned tasks
Client	Invests in projects

1.3 Functional Requirements (User Stories)

1.3.1 Common User Stories

- Users can **Register**
- Users can **Login**
- Users can **View Profile**
- Role-based access control is enforced

1.3.2 Admin Capabilities

- Create Project
- Update Project
- Delete Project
- Assign Project to Manager
- Add / Remove Managers

1.3.3 Project Manager Capabilities

- Assign Project to Builder
- Create Task
- Update Task
- Delete Task
- Assign Task to Builder
- Update Project Status

1.3.4 Builder Capabilities

- View Assigned Projects
- View Assigned Tasks
- Update Task Status

1.3.5 Client Capabilities

- View Available Projects
- Purchase / Invest in Project

- View Purchased Projects
- Request New Project

1.4 System Architecture

Architectural Style: Layered Architecture

Presentation Layer (Console UI)



Service Layer (Business Logic)



Repository Layer (Data Access)



Model Layer (Entities)

Layer Responsibilities

Presentation Layer

- Handles user interaction
- Displays menus
- Accepts input

Service Layer

- Contains business logic
- Performs validation
- Enforces role-based access

Repository Layer

- Handles data storage (in-memory)

- Uses ConcurrentHashMap

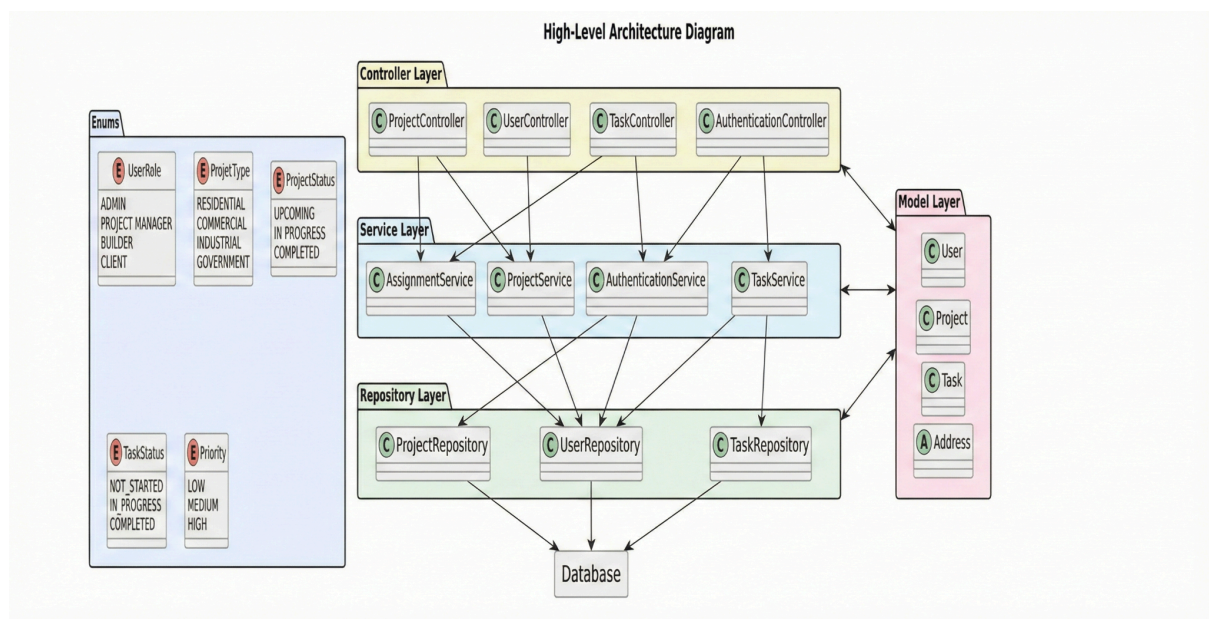
Model Layer

- Contains pure data classes (POJOs)

1.5 Non-Functional Requirements

- Thread Safety using ConcurrentHashMap
- Role-based Authorization
- Modular Design
- Separation of Concerns
- Extensible Architecture

1.6 High Level Architecture Diagram



2.Low-Level Design (LLD)

2.1 Package Structure

models/

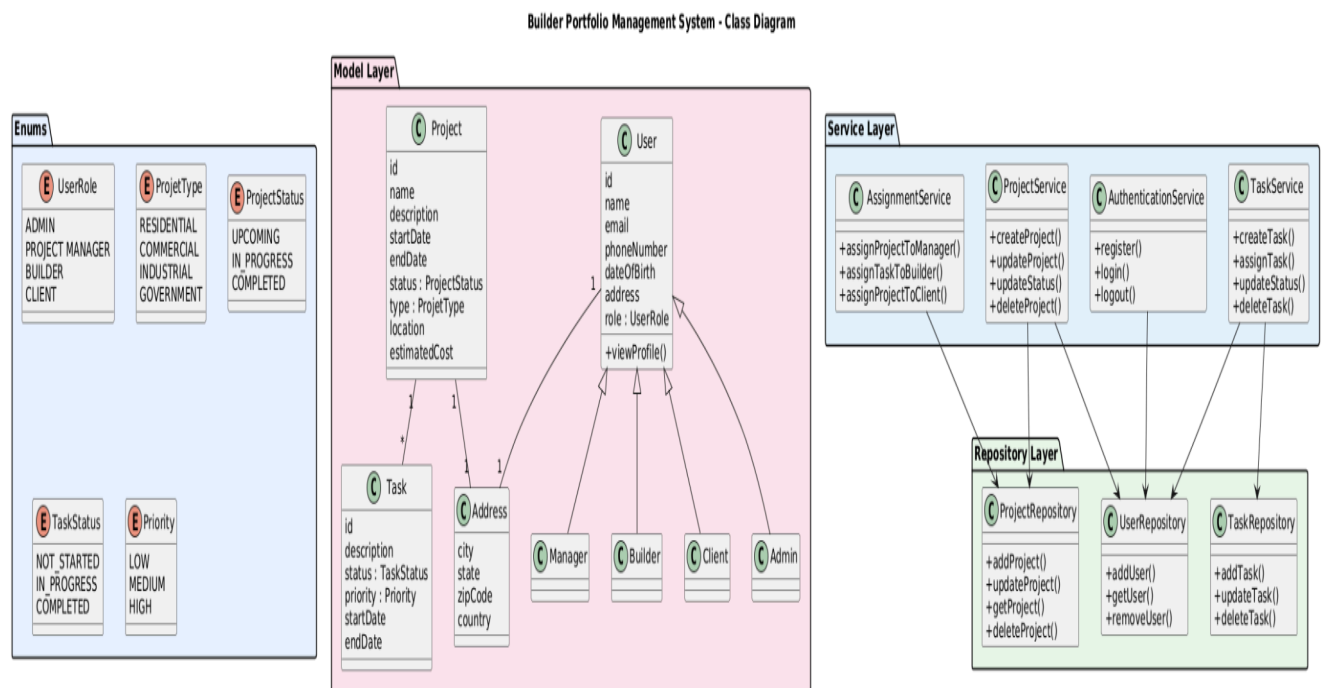
services/

repositories/

consoleUI/

enums/

2.2 UML Diagram



3. Model Layer (Entities)

Entities contain only attributes and simple getters/setters.

3.1 Address

Attributes

- city
- state
- zipCode
- country

3.2 User (Base Class)

Attributes

- id
- name
- address
- phoneNumber
- email
- dateOfBirth
- role (UserRole)

Method

- viewProfile()

3.3 Admin (extends User)

Responsibilities

- Create Project
- Manage Managers
- Assign Project to Manager

3.4 Manager (extends User)

Attributes

- **managingProjects** (ConcurrentHashMap<Project>)
- **managingBuilders** (ConcurrentHashMap<Builder>)

Responsibilities

- **Assign Project to Builder**
- **Create / Update / Delete Tasks**
- **Update Project Status**

3.5 Builder (extends User)

Attributes

- **assignedProjects** (ConcurrentHashMap<Project>)

Responsibilities

- **View Tasks**
- **Update Task Status**

3.6 Client (extends User)

Attributes

- **purchasedProjects** (ConcurrentHashMap<Project>)

Responsibilities

- **Purchase Project**
- **View Purchased Projects**
- **Request New Project**

3.7 Project

Attributes

- **id**

- **name**
- **description**
- **startDate**
- **endDate**
- **status (ProjectStatus)**
- **type**
- **location (Address)**
- **estimatedCost**
- **tasks (ConcurrentHashMap<Task>)**

3.8 Task

Attributes

- **id**
- **description**
- **status (TaskStatus)**
- **startDate**
- **endDate**
- **priority (Priority)**
- **assignedBuilderId**
- **projectId**

3.9 Enums

- **UserRole**
- **ProjectStatus**
- **TaskStatus**

- **Priority**

4.Repository Layer

All repositories use:

`ConcurrentHashMap<String, Entity>`

4.1 UserRepository

- `addUser()`
 - `getUserById()`
 - `getUserByEmail()`
 - `removeUser()`
-

4.2 ProjectRepository

- `addProject()`
- `updateProject()`
- `deleteProject()`
- `getProjectById()`
- `getAllProjects()`

4.3 TaskRepository

- `addTask()`
- `updateTask()`

- `deleteTask()`
- `getTasksByProjectId()`
- `getTasksByBuilderId()`

5. Service Layer

All business logic is implemented in services only.

5.1 AuthenticationService

- `registerUser()`
- `login()`
- `logout()`

5.2 ProjectService

- `createProject()`
- `updateProject()`
- `deleteProject()`
- `viewProject()`
- `updateProjectStatus()`

5.3 TaskService

- `createTask()`
- `updateTask()`
- `deleteTask()`
- `assignTaskToBuilder()`

- `updateTaskStatus()`
-

5.4 AssignmentService

- `assignProjectToManager()`
- `assignProjectToBuilder()`

6. Concurrency Strategy

- `ConcurrentHashMap` for repositories
- Atomic ID generation
- Services act as transactional boundaries
- Role validation before every operation

7. Design Principles Applied

- Single Responsibility Principle (SRP)
- Separation of Concerns
- Encapsulation
- Layered Architecture
- Interface-based service design